ELOWEN HEALTHCARE PLATFORM

Smart Medication Adherence Dashboard

Comprehensive Technical Viva Voce & Project Defense Guide

System Architecture	Decoupled Client-Server SPA with RESTful API & Stateless JWT
Backend Technology	Java 17 LTS, Spring Boot 3.2.5, Spring Security 6, Spring Data JPA
Frontend Technology	React 19, TypeScript 5.7, Tailwind CSS v4, Vite 8 Development Server
Database Engine	H2 Relational Database with Persistent File Storage (./data/adherence_db)
ORM / Mapping	Hibernate 6.4 with Auto DDL schema generation
Authentication	HMAC-SHA256 Signed JSON Web Tokens (JJWT 0.12.5) with BCrypt
Application Server	Embedded Apache Tomcat 10.1 (Built into Spring Boot executable JAR)
Core Innovation	NLP Heuristic Scheduler, Auto Pill Decrement & 30-Day Adherence Analytics

VIVA EXAMINATION PURPOSE & SCOPE

This document covers all theoretical and practical concepts required for project examination: architectural design patterns, authentication mechanics, relational database schemas, algorithm formulation, REST endpoints, and answers to the top 25 viva questions.

1.1 The Clinical Challenge (Medication Non-Adherence)

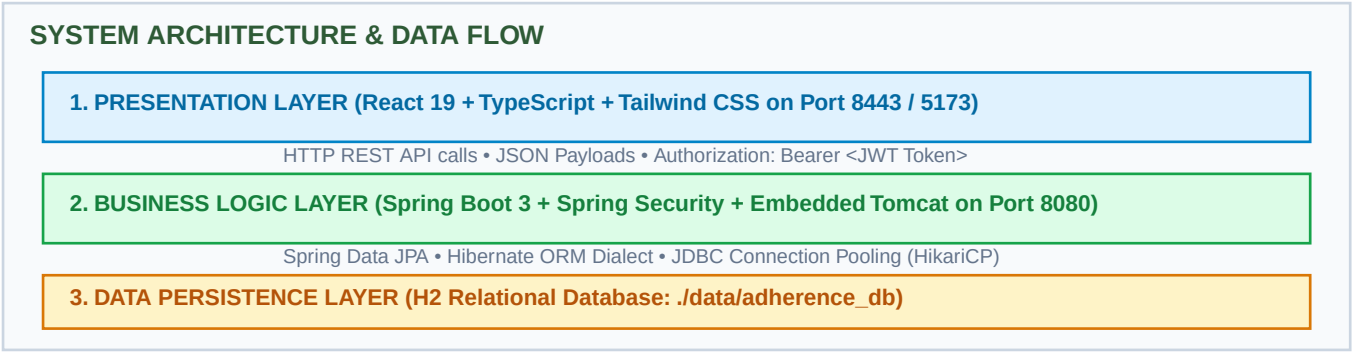
According to World Health Organization (WHO) statistics, over 50% of patients with chronic diseases (such as hypertension, diabetes, and cardiovascular disorders) fail to take their prescribed medicines correctly. Non-adherence leads to severe clinical complications, preventable emergency hospitalizations, and over $100 billion in avoidable healthcare expenditures annually.

1.2 The Elowen Solution

Elowen is a modern full-stack medication management platform combining empathetic patient UX with clinical rigor. It empowers patients to maintain healthy routines while providing caregivers and clinicians with transparent adherence data.

- **Interactive Daily Schedule:** Live dose confirmation with state management (Upcoming, Due Soon, Taken, Snoozed, Missed).
- **Automated Inventory Decrement:** Marking a dose as "Taken" automatically deducts remaining pills and triggers dynamic refill date predictions.
- **Real-Time Clinical Scoring:** Calculates 30-day sliding adherence metrics and consecutive streak days.
- **Natural Language Processing (NLP):** Allows patients to schedule routines via plain English sentences (e.g. "Take Metformin with dinner every weekday at 7:30 PM").
- **Consent-Based Care Circle:** Enables family members and healthcare providers to monitor patient consistency based on permission tiers.

The platform is built on a 3-Tier Layered Architecture with strict decoupling between the Presentation Tier, Application Business Logic Tier, and Data Persistence Tier.



3.1 Stateless JWT Authentication Mechanics

Authentication is implemented without server-side HTTP sessions. When a patient logs in (POST /api/auth/login), Spring Security verifies credentials using BCrypt. Upon success, JJWT generates a digitally signed token using HMAC-SHA256 containing userId, email subject, and a 24-hour expiration timestamp.

For every subsequent request, the frontend sends the token in the Authorization header. Spring Security's JwtAuthenticationFilter intercepts the request, verifies signature integrity, extracts user details, and injects an authenticated principal into the SecurityContextHolder.

3.2 Dose Confirmation & Inventory Transaction Cycle

When a user clicks "Take now", an ACID transaction executes in DoseService (@Transactional):

- **Step 1:** Searches for existing today's log or instantiates a new DoseLog with status = "Taken".
- **Step 2:** Fetches the associated Medication entity and decrements remaining pillCount by 1.
- **Step 3:** Commits the transaction to physical storage in the H2 database.
- **Step 4:** React receives 200 OK, updates dose badges, and triggers a live recalculation of the 30-day adherence rate.

3.3 Natural Language Processing (NLP) Heuristic Engine

Unlike third-party cloud AI APIs that introduce latency and API subscription costs, Elowen includes an internal heuristic NLP scheduling parser in ScheduleService.java:

- **Entity Extraction:** Uses compiled regex patterns to extract explicit times (e.g. "7:30 PM", "8am") and maps meal/lifestyle keywords ("breakfast" -> 8:00 AM, "lunch" -> 1:00 PM, "dinner" -> 7:30 PM).
- **Frequency Tagging:** Identifies routine frequencies such as "weekdays", "weekends", "daily", or "twice daily".
- **Medication Matching:** Cross-references tokens against registered medicines in the database and automatically inserts a new active routine.

3.4 Adherence Calculation Formula

> **Mathematical Formulation**
>
> Adherence Rate (%) = [Total Doses Taken / (Total Taken + Total Missed + Total Skipped)] * 100
>
> Current Streak = Consecutive daily sequence where no scheduled dose is in "MISSED" status.
> Refill Prediction = Today + (Remaining Pills / Daily Dose Frequency) Days.

4.1 Relational Database Schema (H2 Persistent Engine)

USERS	id (PK), email (UNIQUE), password (BCrypt hash), name, timezone, push_notifs, email_reports, caregiver_alerts
MEDICATIONS	id (PK), user_id (FK), name, dosage, detail, pill_count, total_capacity, color, icon, schedule_time, refill_date
DOSE_LOGS	id (PK), user_id (FK), medication_id (FK), medication_name, scheduled_time, dose_date, status, notes, logged_at
SCHEDULES	id (PK), user_id (FK), title, frequency, time_str, medication_name, active
CARE_CIRCLE	id (PK), user_id (FK), name, relationship, status (Active/Pending), email, phone

4.2 Core REST API Endpoint Reference

POST	**/api/auth/login**	Authenticates email & password; returns JWT token & UserDto profile.
POST	**/api/auth/register**	Creates new user account with BCrypt password hashing.
POST	**/api/auth/google**	One-click instant authentication for demo evaluation.
GET	**/api/dashboard/overview**	Aggregated live metrics: adherence %, streak, today doses, trend chart, stock.
GET/POST	**/api/medications**	List user medications or register a new medicine with pill inventory.
DELETE	**/api/medications/{id}**	Removes medication and cascade-cleans associated references.
POST	**/api/doses/action**	Records dose (TAKEN/SNOOZED/MISSED) and auto-decrements stock.
GET	**/api/doses/history**	Full historical dose log for CSV audit export.
POST	**/api/schedules/natural-language**	Submits sentence prompt to NLP engine; outputs created schedule.
GET/POST	**/api/care-circle**	Manages caregivers, clinician contacts, and permission levels.
PUT	**/api/user/profile**	Updates patient name, email, or preferred timezone.
PUT	**/api/user/notifications**	Persists notification toggles (Push, Email, Caregiver).

Spring Boot eliminates XML and manual deployment descriptors (web.xml). It offers opinionated starter dependencies (spring-boot-starter-web, spring-boot-starter-data-jpa) and bundles an embedded Apache Tomcat server directly into an executable JAR. This provides rapid deployment and reproducible execution.

Q2: How does the application implement stateless authentication?

The server maintains zero session state in memory. Upon login, JJWT issues a cryptographically signed HMAC-SHA256 token containing identity claims and expiration. Clients include this token in the "Authorization: Bearer <token>" header. JwtAuthenticationFilter validates the cryptographic signature on every HTTP request.

Q3: Why use BCrypt for password hashing instead of MD5 or SHA-256?

MD5 and SHA-256 are fast general-purpose hash algorithms, making them vulnerable to rainbow table attacks and GPU-accelerated brute force. BCrypt incorporates automatic salt generation and a configurable cost factor (work factor 10), requiring significant CPU time per attempt to thwart dictionary attacks.

Q4: Is the H2 database running in-memory or on disk? How is persistence guaranteed?

It is configured in file-persistent mode: "jdbc:h2:file:./data/adherence_db;DB_CLOSE_DELAY=-1". Unlike in-memory H2 (jdbc:h2:mem:...) which wipes data when the JVM terminates, file-based H2 persists all tables, patient records, and dose logs onto physical disk storage.

Q5: How does the frontend communicate with the backend during development? What is CORS?

React runs on Vite (port 8443) while Spring Boot runs on port 8080. Browsers enforce the Same-Origin Policy (SOP). We enabled Cross-Origin Resource Sharing (CORS) in Spring Security via a CorsConfigurationSource bean and added a Vite proxy in vite.config.ts routing /api requests directly to localhost:8080.

Q6: What happens under the hood when a patient marks a dose as "Taken"?

A POST request hits /api/doses/action. Spring's DoseService runs inside a @Transactional boundary: it updates/creates a DoseLog entry with status "Taken", fetches the Medication entity, decrements its remaining pillCount by 1, and commits the transaction. The UI immediately recalculates KPIs and streak metrics.

The NLP engine in ScheduleService.java uses regex pattern matching and semantic dictionaries. It scans for explicit times ("8:00 AM", "7:30 PM") and time-of-day phrases ("morning", "lunch", "dinner"). It then detects recurrence frequencies ("weekdays", "daily") and matches medicine names against the user's database inventory.

Q8: What is the distinction between JPA and Hibernate?

JPA (Jakarta Persistence) is a standardized Java specification that defines interfaces and annotations (@Entity, @Id, @ManyToOne). Hibernate is the actual Object-Relational Mapping (ORM) provider that implements the JPA specification and handles raw SQL translation and JDBC connections.

Q9: What architectural design patterns are utilized in this project?

1) Controller-Service-Repository Pattern: Strict 3-tier layering.
2) DTO (Data Transfer Object) Pattern: Decouples DB entities from REST payloads.
3) Interceptor/Filter Pattern: JwtAuthenticationFilter intercepts requests before controllers.
4) Dependency Injection (IoC): Managed by Spring ApplicationContext.
5) Singleton Pattern: Spring beans are singletons by default.

Q10: How does Spring Data JPA execute queries without explicit SQL implementations?

Spring Data JPA uses dynamic proxy generation at startup. When declaring repository interfaces extending JpaRepository, Spring inspects method names (such as findByEmail or countByUserIdAndStatus) and generates the corresponding JPQL/SQL queries automatically.

Q11: What is the function of the DataInitializer class?

DataInitializer implements Spring Boot's CommandLineRunner interface. When the app boots, if the database is empty, it automatically populates demo patient accounts, medications, schedules, and 30 days of dose history so the dashboard displays live analytics immediately.

Q12: How does the frontend handle token storage and security?

JWT tokens and sanitized user profile objects are stored in the browser's localStorage. On every protected API call, the api.ts client attaches the token to the Authorization header. On HTTP 401 unauthorized errors, stored tokens are cleared and the user is redirected to the login view.

1) Database: Migrate from H2 to PostgreSQL/MySQL with read-replicas.
2) Caching: Deploy Redis to cache dashboard metrics and user profiles.
3) Connection Pooling: Fine-tune HikariCP parameters for high concurrency.
4) Asynchronous Messaging: Use RabbitMQ/Apache Kafka to trigger SMS/Email caregiver alerts.
5) Deployment: Package as Docker containers deployed on Kubernetes (EKS/GKE).

Q14: How does Spring Security protect against CSRF attacks in this application?

Cross-Site Request Forgery (CSRF) relies on ambient credentials (browser cookies sent automatically). Because our API is completely stateless and uses custom Authorization: Bearer headers stored in localStorage, browsers do not automatically attach credentials, making CSRF protection unnecessary (csrf.disable()).

Q15: What is the difference between PUT and POST HTTP verbs in your API?

POST is used for creating new resources (e.g. POST /api/medications, POST /api/auth/register). PUT is idempotent and used for updating/replacing existing resources (e.g. PUT /api/user/profile, PUT /api/user/notifications).

7.1 Examiner Defense Strategy (How to Answer During Viva)

- **Clarity Over Jargon:** State the "Why" before the "What". For example, explain *why* JWT was used (stateless horizontal scaling) before explaining how it was configured.
- **Demonstrate Database Persistence:** If asked whether data is stored in memory, emphasize that H2 is configured in persistent file mode at ./data/adherence_db.
- **Highlight Full-Stack Cohesion:** Mention how taking a dose on the React UI sends a REST request to Spring Boot, triggers an ACID transaction in DoseService, decrements inventory, and dynamically refreshes the adherence percentage.

PROJECT DEMO CREDENTIALS: alex@example.com / password123 | H2 Console: http://localhost:8080/h2-console